

Compiler Optimizations for Higher-Order Automatic Differentiation in JAX

Anonymous Authors¹

Abstract

Higher-order automatic differentiation (AD) is increasingly important in scientific machine learning, but mathematically equivalent methods for computing Hessians, Hessian-vector products, and Laplacians can lower to substantially different tensor programs. We study this effect in JAX by comparing Hessian, JVP-of-gradient, and Taylor-mode AD formulations, lowering each to StableHLO, and dispatching both original and transformed programs through XLA. Our pipeline combines a direct tanh-MLP Laplacian rewrite with StableHLO constant propagation and structural-zero elimination. On MLP Laplacian and Poisson PINN benchmarks, optimization reduces average runtime by up to $1.94\times$ and compile time by up to $2.28\times$. The fastest AD method before optimization is not always the fastest after optimization, suggesting that higher-order AD method selection should account for downstream compiler optimizability.

1. Introduction

Automatic differentiation (AD) has become a central tool in machine learning, particularly in deep learning, enabling the training of complex models in frameworks like PyTorch and TensorFlow. AD computes derivatives by systematically applying the chain rule to the primitive operations of a program, producing derivatives that are exact up to floating-point arithmetic. While most widely used AD systems are optimized for first-order derivatives, many emerging applications require second- and higher-order derivatives, including Hessians, Jacobian-vector products, and Laplacians.

Higher-order AD is expensive, but its cost depends not only on the derivative being computed, but also on how the derivative program is generated and optimized. Different higher-order AD strategies, such as nested first-order

AD (e.g. repeatedly applying first-order forward/reverse mode AD) or Taylor-mode AD (e.g. computing higher-order derivatives via a Taylor series expansion), can produce mathematically equivalent derivatives while lowering to structurally different tensor programs. These differences affect the opportunities available to downstream compiler optimizations.

In this paper, we study higher-order AD as a compiler optimization problem. We lower several higher-order AD methods to StableHLO, apply several optimization passes, and execute both original and optimized programs through XLA. We make the following contributions:

1. We present a pipeline for comparing higher-order AD methods before and after compiler optimization.
2. We evaluate repeated first-order AD compositions and Taylor-mode AD on higher-order derivative workloads.
3. We show that compiler optimization can substantially improve performance, reducing runtime by up to $1.94\times$ and compile time by up to $2.28\times$ on Laplacian benchmarks.
4. We demonstrate that compiler optimizations can change which AD strategy performs best, suggesting that AD method selection should account for downstream compiler optimizability.

2. Higher-Order AD Methods Produce Different Compiler IR

In general, AD constructs a new computation that evaluates derivatives of the original computation. For example, suppose we write a simple JAX function:

```
def f(x):  
    return x * x + 1.0  
  
g = jax.grad(f)
```

Mathematically, $g(x) = 2x$. However, from the compiler's point of view, `jax.grad(f)` does not simply record this symbolic formula; instead, it constructs a new program whose operations compute the derivative of `f`. This distinction becomes especially important for higher-order AD: two

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

methods may compute the same mathematical derivative while generating substantially different compiler IR.

In JAX, higher-order derivatives can be computed using transformations such as `jax.hessian`, `jax.jvp`, `jax.grad`, or `jax.experimental.jet`. These transformed programs can be traced into JAX’s internal *jaxpr* representation, a functional IR that exposes the primitive operations and data dependencies of the staged computation. JAX then lowers *jaxpr* to StableHLO, a portable tensor-program IR in the OpenXLA ecosystem. StableHLO is then passed to XLA, which produces an executable for CPU, GPU, or TPU backends. StableHLO represents the generated computation using high-level operations such as `dot_general`, `add`, `multiply`, `tanh`, `reshape`, `broadcast`, and `reduce`.

Crucially, the choice of higher-order AD method determines the StableHLO structure exposed to the compiler, and therefore which optimizations become available. Below, we discuss several higher-order AD methods.

2.1. Nested First-Order AD

A common strategy for computing higher-order derivatives is to repeatedly compose first-order AD transformations. This approach is appealing because it reuses mature first-order AD infrastructure: second-order quantities can be computed by differentiating a program that already computes a derivative.

The drawback is that the generated derivative program may be much larger than the final quantity of interest. Since each AD transformation is applied to the program produced by the previous one, nested AD can duplicate intermediate computations and fail to expose higher-order structure such as symmetry or sparsity. For example, when computing a Laplacian, only the diagonal entries of the Hessian are needed, but a nested AD formulation may still generate IR resembling a larger Hessian computation. This leads to redundant tensor operations and increased compilation overhead.

2.1.1. HESSIAN

Take computing the Laplacian (the trace of the Hessian) of a function, for example. One approach is to first materialize the full $n \times n$ Hessian via `jax.hessian`, which is implemented as `jacfwd(jacrev(f))`, and then take its trace:

```
hessian = jax.hessian(scalar_fn)(x)
return jnp.trace(hessian)
```

Mathematically, this constructs the full matrix (left) and

returns the following (right):

$$H(x) = \nabla^2 f(x) \in \mathbb{R}^{d \times d} \quad \text{tr}(H(x)) = \sum_i H_{ii}(x).$$

At the IR level, this method materializes a Hessian-shaped tensor before applying an explicit trace. This exposes a recognizable trace pattern to the compiler: a compiler pass could use this structure to canonicalize the trace operation and remove off-diagonal computations that do not contribute to the result.

2.1.2. HESSIAN-VECTOR PRODUCT

Another way to compute the Laplacian is to use Hessian-vector products rather than explicitly invoking a Hessian transformation. Hessian-vector products are a specialized instance of nested first-order AD. In particular, `jax.jvp(jax.grad(f), (x,), (v,))` computes a Hessian-vector product using forward-over-reverse AD: reverse mode constructs the gradient computation, and forward mode differentiates that gradient computation in the direction v .

For a direction vector v , consider the following:

```
grad_fn = jax.grad(scalar_fn)
_, hvp = jax.jvp(grad_fn, (x,), (v,))
return jnp.dot(v, hvp)
```

Since the Jacobian of ∇f is the Hessian, the tangent output of this JVP is

$$\text{JVP}(\nabla f)(x; v) = \nabla^2 f(x)v.$$

Taking the inner product with the same direction gives the second directional derivative

$$v^\top \nabla^2 f(x)v.$$

When v is a standard basis vector e_i , this quantity is exactly the i -th diagonal entry of the Hessian:

$$e_i^\top \nabla^2 f(x)e_i = H_{ii}(x).$$

Thus, evaluating this computation over all basis directions and summing the results yields the Laplacian,

$$\Delta f(x) = \sum_i e_i^\top \nabla^2 f(x)e_i.$$

2.2. Taylor-Mode AD

An alternative strategy computes higher-order derivatives via truncated Taylor series, implemented in JAX as `jax.experimental.jet`. For each basis direction e_i , Taylor-mode considers the scalar curve

$$g_i(t) = f(x + te_i).$$

and extracts

$$g_i''(0) = e_i^\top \nabla^2 f(x) e_i = H_{ii}(x).$$

Summing these second derivatives over all basis directions gives the Laplacian:

$$\Delta f(x) = \sum_i H_{ii}(x).$$

At the IR level, Taylor-mode exposes explicit Taylor-coefficient propagation. This introduces coefficient-normalization operations, such as `chlo.lgamma` terms, as well as many structurally zero Taylor components, creating opportunities for constant folding, zero propagation, and dead-code elimination.

3. Optimization Passes

We ask ourselves the following research question:

How do we effectively optimize code which requires the use of higher-order derivatives?

We believe that the answer to this question lies in designing or adapting compiler techniques can exploit the high-level structure exposed by derivative operators. Specifically, we implement the following passes described below. Figure 1 shows the full pipeline of where our optimizations lie.

Direct Input-Laplacian Recurrence for Tanh MLPs.

Our first optimization targets the scalar tanh-MLP Laplacian benchmark. Rather than lowering a generic higher-order AD expression such as `hessian(f)`, repeated `jvp` applications, or Taylor-mode AD, we generate code for a direct forward recurrence that computes the input Laplacian of the network.

For each layer, the ordinary forward pass computes

$$z_\ell = a_{\ell-1} W_\ell + b_\ell, \quad a_\ell = \tanh(z_\ell).$$

The optimized recurrence augments this forward pass with two input-derivative quantities: the Jacobian

$$G_\ell = \frac{\partial a_\ell}{\partial x}$$

and the componentwise input Laplacian

$$\lambda_\ell = \Delta a_\ell.$$

Thus each layer propagates the triple

$$(a_\ell, G_\ell, \lambda_\ell),$$

where a_ℓ is the activation, G_ℓ records first derivatives with respect to the original input, and λ_ℓ records the Laplacian of each activation component.

The recurrence is initialized by

$$a_0 = x, \quad G_0 = I, \quad \lambda_0 = 0.$$

For the affine part of layer ℓ , derivatives propagate linearly:

$$G_\ell^z = G_{\ell-1} W_\ell, \quad \lambda_\ell^z = \lambda_{\ell-1} W_\ell.$$

The tanh nonlinearity is then handled by the chain rule:

$$a_\ell = \tanh(z_\ell),$$

$$G_\ell = \tanh'(z_\ell) \odot G_\ell^z,$$

and

$$\lambda_\ell = \tanh''(z_\ell) \odot \|G_\ell^z\|_{\text{row}}^2 + \tanh'(z_\ell) \odot \lambda_\ell^z.$$

Here $\|G_\ell^z\|_{\text{row}}^2$ denotes the squared norm of the input-gradient vector for each activation component. Using $\tanh'(z) = 1 - \tanh^2(z)$ and $\tanh''(z) = -2 \tanh(z)(1 - \tanh^2(z))$, the recurrence lowers to ordinary tensor operations such as `dot_general`, `tanh`, `multiply`, `add`, and `reduce`.

This computes the same input Laplacian as the corresponding higher-order AD program, but avoids materializing the full Hessian or constructing a large nested derivative computation.

Constant Propagation Higher-order AD, especially Taylor-mode AD, can introduce scalar coefficient computations that are independent of the program input. In the StableHLO emitted by `jax.experimental.jet`, these appear as constant `chlo.lgamma` calls, followed by `exponential`, `convert`, `broadcast_in_dim`, and arithmetic operations used to scale Taylor coefficients. Our constant propagation pass evaluates such scalar and splat constant expressions before dispatching the program to XLA.

For example, a derivative-generated StableHLO program may contain:

```
// Before constant propagation
%cst = stablehlo.constant
      dense<1.00e+01> : tensor<f32>
%0 = chlo.lgamma %cst
     : tensor<f32> -> tensor<f32>
```

The pass rewrites this to:

```
// After constant propagation
%0 = stablehlo.constant
     dense<1.280183e+01> : tensor<f32>
```

The same pass also folds chains of constant operations. For example, Jet often emits coefficient-normalization code of the form:



Figure 1. Compilation pipeline showing where our compiler passes are inserted. The derivative-level pass rewrites the higher-order derivative computation before lowering, while the StableHLO passes optimize the lowered IR before dispatching to XLA.

```

175 %cst = stablehlo.constant
176     dense<3.00e+00> : tensor<f32>
177 %0 = chlo.lgamma %cst
178     : tensor<f32> -> tensor<f32>
179 %1 = stablehlo.exponential %0
180     : tensor<f32>
181 %2 = stablehlo.broadcast_in_dim %1,
182     dims = []
183     : (tensor<f32>)
184     -> tensor<256x3xf32>
    
```

After constant propagation, the `lgamma` and `exponential` operations are replaced by constants, leaving a simpler StableHLO program with fewer Taylor-mode bookkeeping operations.

Structural Zero Elimination Higher-order AD also introduces tangent, cotangent, or Taylor-coefficient values that are zero by construction. These zeros can be broadcast, reshaped, and propagated through arithmetic before later passes remove them. Our structural zero elimination pass tracks syntactically known zero SSA values in StableHLO and rewrites operations whose results are also known to be zero.

For example, Jet-generated code often contains broadcasted zero tensors that are immediately added into a derivative computation:

```

204 // Before structural zero elimination
205 %z = stablehlo.constant
206     dense<0.00e+00> : tensor<f32>
207 %0 = stablehlo.broadcast_in_dim %z,
208     dims = []
209     : (tensor<f32>)
210     -> tensor<256x3x256xf32>
211 %1 = stablehlo.add %0, %x
212     : tensor<256x3x256xf32>
    
```

Since `%0` is known to be zero, the addition can be replaced by an alias to `%x`:

```

217 // After structural zero elimination
218 %1 = %x
    
```

4. Evaluation

We evaluate the optimization passes on two higher-order differentiation workloads implemented in JAX: an input-space Laplacian of a scalar MLP and a Poisson physics-informed neural network (PINN). In both benchmarks, we compare three mathematically equivalent ways of computing the required second derivatives: `hessian`, `jvp_grad`, and `jet`. We do not include the automatic schedule-selection path in this paper; all reported methods are fixed source-level AD formulations.

Methods. The `hessian` method computes the dense Hessian of the scalar function with `jax.hessian` and takes the trace. The `jvp_grad` method forms Hessian-vector products by applying `jax.jvp` to `jax.grad`; for each standard basis direction e_i , it computes $e_i^T \nabla^2 f(x) e_i$, then sums over i . The `jet` method uses `jax.experimental.jet` to propagate second-order Taylor coefficients along each basis direction and sums the resulting second directional derivatives. These methods produce the same mathematical Laplacian, but they expose different StableHLO programs to the compiler.

Benchmarking protocol. For each benchmark and method, the Python benchmark lowers the JAX function to StableHLO, compiles the original StableHLO through XLA/PJRT, runs untimed warmup executions, and then records timed executions. The default timing configuration uses three warmup runs and thirty measured runs. To test how the methods scale with model width, we sweep the hidden dimension

$$h \in \{128, 256, 384, 512\}$$

for both benchmarks while holding the number of layers, input dimension, number of evaluation points, seed, dtype, and AD method fixed. We report JIT compile time and steady-state execution time for the original StableHLO and for the StableHLO after our optimization-pass pipeline. The benchmark scripts also record the backend, output summary, and pass-level compiler statistics. In the tables below, the compile-time column reports original StableHLO compile time, the after-pass compile-time column reports compile time for the transformed StableHLO, and the runtime

columns report average execution time across the measured runs.

4.1. MLP Laplacian

The first benchmark computes the input Laplacian of a scalar fully connected tanh MLP. The network has input dimension 3, output dimension 1, and 128 hidden layers. We vary the hidden width $h \in \{128, 256, 384, 512\}$. Each hidden layer applies an affine map followed by tanh; the final layer is linear. We initialize weights from a normal distribution scaled by $1/\sqrt{\text{fan-in}}$, use zero biases, and store all parameters in `float32`. The benchmark uses seed 0.

The input batch contains 256 points in $\mathbb{R}^3$. In the code, these points are generated deterministically by linearly spacing 256×3 `float32` values between -1 and 1 , then reshaping them into a 256×3 matrix. For each point x , the task computes

$$\Delta f_{\theta}(x) = \text{tr}(\nabla_x^2 f_{\theta}(x)),$$

and the benchmark returns the vector of 256 Laplacian values. This workload isolates input-derivative computation: the neural network parameters are fixed, and no outer parameter-gradient computation is performed.

4.2. Poisson PINN

The second benchmark evaluates a Poisson PINN residual and differentiates the loss with respect to network parameters. The neural network again uses 128 hidden tanh layers, but the input dimension is 2 and the output dimension is 1. We use the same hidden-width sweep $h \in \{128, 256, 384, 512\}$, with the same seed-0, `float32`, fan-in-scaled normal weight initialization and zero-bias initialization.

The PINN defines a trial solution

$$u_{\theta}(x, y) = x(1 - x)y(1 - y)N_{\theta}(x, y),$$

where the multiplicative boundary factor enforces zero boundary values on the unit square. Collocation points are a 16×16 tensor-product grid, giving 256 points total, with each coordinate linearly spaced from 0.1 to 0.9. At each collocation point, the benchmark computes the residual

$$r_{\theta}(x, y) = \Delta u_{\theta}(x, y) + \sin(\pi x) \sin(\pi y).$$

The scalar loss is

$$\mathcal{L}(\theta) = \frac{1}{256} \sum_{(x,y)} r_{\theta}(x, y)^2,$$

and the benchmarked JAX program returns both $\mathcal{L}(\theta)$ and $\nabla_{\theta} \mathcal{L}(\theta)$ via `jax.value_and_grad`. Thus, unlike the MLP Laplacian benchmark, this workload nests input-space second derivatives inside an outer reverse-mode parameter-gradient computation.

5. Discussion

5.1. Compiler Optimization Can Change the Best AD Method

The results show that choosing an AD method before compiler optimization is not equivalent to choosing one after optimization. In the unoptimized StableHLO, the fastest method depends on the derivative formulation and on the benchmark. For example, in the MLP Laplacian benchmark, `jet` is fastest at hidden width 128, but `jvp_grad` is fastest for widths 256, 384, and 512. In the Poisson PINN benchmark, `hessian` is fastest at width 128, while `jvp_grad` is fastest for the larger widths before our passes are applied.

After optimization, the ranking changes in several cases. For the MLP Laplacian at width 256, the fastest method changes from `jvp_grad` before optimization to `jet` after optimization. For the Poisson PINN, the changes are more pronounced: at width 256, the fastest method changes from `jvp_grad` to `jet`, while at widths 384 and 512, it changes from `jvp_grad` to `hessian`. Thus, the method that gives the best initial compiler IR is not necessarily the method that gives the best optimized executable.

This behavior illustrates why higher-order AD performance should be evaluated jointly with compiler optimization. A source-level AD transform determines the initial IR, but the compiler may expose, remove, or replace different parts of that IR. In our benchmarks, the direct tanh-MLP Laplacian recurrence removes much of the structural difference between the original Hessian, JVP-of-gradient, and Taylor-mode formulations. As a result, after-pass runtimes are often much closer to each other than before-pass runtimes, especially in the PINN benchmark. The remaining differences then reflect a combination of residual IR structure, compile decisions, and backend execution effects rather than only the source-level AD formulation.

5.2. Limitations and Future Work

This work is a prototype evaluation of compiler optimizations for higher-order AD, and its scope is intentionally narrow. The direct Laplacian recurrence is specialized to scalar tanh MLPs with affine layers, and the evaluation fixes the input dimension, batch size, number of layers, dtype, and random seed while sweeping only hidden width. The Poisson PINN benchmark is more representative of scientific machine learning because it nests input-space second derivatives inside an outer parameter-gradient computation, but it is still a single PDE, single network family, and single collocation-grid shape. We also evaluate a single backend configuration; GPU and TPU backends may expose different tradeoffs between larger derivative IR, backend fusion, memory traffic, and compile time.

Our StableHLO passes are also conservative. Constant prop-

Table 1. MLP Laplacian results under a hidden-width sweep. Runtime columns are average runtime over 30 measured executions.

Hidden width	Method	Compile before (ms)	Compile after (ms)	Runtime before (ms)	Runtime after (ms)	Speedup
128	hessian	330.9	222.0	54.3	31.0	1.75
	jvp_grad	286.2	222.0	54.9	30.9	1.77
	jet	404.2	254.3	45.1	30.4	1.49
256	hessian	256.5	176.9	101.6	60.0	1.69
	jvp_grad	240.7	176.3	95.4	63.9	1.49
	jet	379.0	179.4	105.4	59.8	1.76
384	hessian	268.5	185.5	183.0	112.2	1.63
	jvp_grad	237.5	183.3	180.4	111.6	1.62
	jet	430.2	183.4	215.5	125.4	1.72
512	hessian	323.3	186.2	308.4	195.3	1.58
	jvp_grad	238.9	176.1	295.3	193.9	1.52
	jet	391.6	176.8	379.6	195.7	1.94

Table 2. Poisson PINN results under a hidden-width sweep. Runtime columns are average runtime over 30 measured executions.

Hidden width	Method	Compile before (ms)	Compile after (ms)	Runtime before (ms)	Runtime after (ms)	Speedup
128	hessian	1770.7	859.7	107.9	76.9	1.40
	jvp_grad	1436.3	809.1	110.8	77.3	1.43
	jet	1599.1	823.4	128.8	77.1	1.67
256	hessian	1727.9	1030.0	217.3	176.3	1.23
	jvp_grad	1458.8	813.2	214.7	158.2	1.36
	jet	1602.4	763.1	252.0	155.5	1.62
384	hessian	1670.4	822.4	434.5	282.5	1.54
	jvp_grad	1420.8	807.8	413.6	283.9	1.46
	jet	1627.7	797.3	515.9	282.9	1.82
512	hessian	1786.3	783.6	711.0	462.7	1.54
	jvp_grad	1537.8	788.6	704.9	465.8	1.51
	jet	1612.7	781.5	844.6	465.4	1.82

agation and structural zero elimination target patterns that appear in the generated IR for the benchmarks studied here, and the direct recurrence relies on recognizing a high-level mathematical structure before lowering. We do not yet provide a general optimizer for arbitrary higher-order derivative programs, nor do we prove equivalence for every possible StableHLO rewrite. In particular, broader support for other activations, vector-valued outputs, convolutional networks, non-MLP PDE solvers, and mixed precision would require additional rewrite rules and validation.

Future work should extend the pass pipeline in three directions. First, the compiler should exploit higher-order derivative identities such as mixed-partial symmetry, Hessian sparsity, and diagonal-only access patterns. These identities could enable common subexpression elimination

across derivative subgraphs and kernel rewrites that avoid forming unused tensor entries. Second, the current benchmark suite should be expanded to include additional PDEs, larger input dimensions, hardware backends, and end-to-end training loops. Third, the passes should be integrated more tightly with the JAX/StableHLO lowering path so that the optimization opportunities are available during normal compilation rather than through an offline experimental pipeline.

5.3. Related Work

Automatic differentiation systems expose derivatives through a range of program transformation mechanisms. JAX provides composable transformations such as `grad`, `jvp`, and `hessian` over NumPy-style programs (Bradbury et al., 2018), while PyTorch popularized imperative tensor

programs with high-performance reverse-mode AD for deep learning (Paszke et al., 2019). Classical AD treatments describe the algorithmic foundations of forward, reverse, and higher-order modes (Griewank & Walther, 2008). Taylor-mode AD computes higher-order derivatives by propagating truncated Taylor coefficients; JAX Jet implements this idea for JAX programs and avoids some redundant work that arises from naively composing first-order transformations (Bettencourt et al., 2019). Our evaluation compares Jet against two nested first-order formulations after lowering to StableHLO, emphasizing that their relative performance depends on the downstream compiler as well as on the source-level AD algorithm.

Several systems move AD into the compiler. Enzyme is a high-performance automatic differentiation compiler plugin for LLVM that differentiates optimized LLVM IR and targets programs written in languages that lower to LLVM (Moses & Churavy, 2020). Later Enzyme work extends this compiler-based approach to reverse-mode AD for GPU kernels and multiple parallel programming paradigms (Moses et al., 2021; 2022). Enzyme is complementary to this paper: it synthesizes derivative programs at the compiler IR level, while our prototype starts from derivative programs already produced by JAX and optimizes the StableHLO tensor IR before XLA compilation.

Compiler optimization for tensor programs is also closely related. MLIR provides the extensible compiler infrastructure underlying many domain-specific IRs, including tensor compiler pipelines (Lattner et al., 2021). TASO shows that tensor computation graphs can be optimized through automatically generated graph substitutions (Jia et al., 2019). Most closely related to this work, Sathia et al. study compiler optimizations for higher-order AD in StableHLO and argue that derivative-generated tensor programs contain exploitable structure such as constants, zeros, symmetry, and sparse access patterns (Sathia et al., 2025). Our paper follows this compiler-oriented view, but adds a more detailed evaluation across both standalone MLP Laplacians and a Poisson PINN workload. PINNs use neural networks to represent PDE solutions and rely on differentiating network outputs with respect to physical coordinates (Raissi et al., 2019); this makes them a natural setting for testing whether compiler optimizations improve second-order AD in scientific machine learning.

6. Conclusion

Higher-order AD programs are not only mathematical derivative computations; they are also compiler inputs. We showed that three equivalent JAX formulations for Laplacian computations, hessian, jvp-grad, and jet, lower to different StableHLO programs and can respond differently to compiler optimization. A small optimization

pipeline for tanh-MLP Laplacians and derivative-generated StableHLO reduces both compile time and steady-state runtime on MLP Laplacian and Poisson PINN benchmarks.

The main empirical lesson is that the best AD method can change after compiler optimization. In several settings, the formulation with the fastest original runtime is not the one with the fastest optimized runtime. This suggests that higher-order AD systems should not treat derivative generation and compiler optimization as separate concerns. Instead, AD methods should be evaluated by the optimized executable they produce, and compilers should expose passes that recognize the structure introduced by higher-order differentiation.

References

- Bettencourt, J., Johnson, M. J., and Duvenaud, D. Taylor-mode automatic differentiation for higher-order derivatives in JAX. In *Program Transformations for Machine Learning Workshop at NeurIPS 2019*, 2019. URL <https://openreview.net/forum?id=SkxEF3FNPH>.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.
- Griewank, A. and Walther, A. *Evaluating Derivatives*. Society for Industrial and Applied Mathematics, 2nd edition, 2008. doi: 10.1137/1.9780898717761.
- Jia, Z., Padon, O., Thomas, J., Warszawski, T., Zaharia, M., and Aiken, A. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pp. 47–62, 2019. doi: 10.1145/3341301.3359630.
- Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., and Zinenko, O. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization*, pp. 2–14, 2021. doi: 10.1109/CGO51591.2021.9370308.
- Moses, W. S. and Churavy, V. Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients. In *Advances in Neural Information Processing Systems*, volume 33, pp. 12472–12485, 2020. URL <https://proceedings.neurips.cc/paper/2020/file/9332c513ef44b682e9347822c2e457ac-Paper.pdf>.

- Moses, W. S., Churavy, V., Paehler, L., Hückelheim, J., Narayanan, S. H. K., Schanen, M., and Doerfert, J. Reverse-mode automatic differentiation and optimization of GPU kernels via Enzyme. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16, 2021. doi: 10.1145/3458817.3476165.
- Moses, W. S., Narayanan, S. H. K., Paehler, L., Churavy, V., Schanen, M., Hückelheim, J., Doerfert, J., and Hovland, P. Scalable automatic differentiation of multiple parallel paradigms through compiler augmentation. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pp. 1–18, 2022.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pp. 8024–8035, 2019.
- Raissi, M., Perdikaris, P., and Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- Sathia, V., Qian, S. B., Hückelheim, J., Hovland, P., and Moses, W. S. Compiler optimizations for higher-order automatic differentiation. In *Proceedings of the Workshop on Differentiable Parallel Programming*, Las Vegas, NV, USA, 2025. ACM. URL <https://ece.is/assets/pdf/diffprog-ppopp25-paper90.pdf>.